

第 13 章 PCIe 总线与虚拟化技术

目前虚拟化技术在处理器体系结构中，已经占据一席之地。虚拟化技术由来已久，其含义也较为广泛，多个进程共享一个 CPU，多个进程的虚拟空间共享同一个物理内存等一系列在体系结构中已经根深蒂固的概念，都可以归于虚拟化技术。

本章所强调的虚拟化技术是指在一个处理器系统^①中运行多个虚拟处理器系统的技术。其中每一个虚拟处理器系统都有独立的虚拟运行环境，包括 CPU、内存和外部设备。在这个虚拟环境中运行的操作系统彼此独立，但是这些操作系统仍使用相同的物理资源。

因此处理器需要为虚拟化环境设置专门的硬件，以支持多个虚拟处理器系统在一个物理环境中的资源共享。虚拟化技术的核心是通过 VMM（Virtual Machine Monitor）集中管理物理资源，而每个虚拟处理器系统通过 VMM 访问实际的物理资源。有时为了提高虚拟机访问外部设备的效率，虚拟处理器系统也可以直接访问物理资源。

在一个处理器系统中，这些物理资源包括 CPU、主存储器、外部设备和中断。IA 处理器^②使用 EPT（Extended Page Table）和 VPID 技术对主存储器进行管理，而使用虚拟中断控制器接管中断请求以实现中断的虚拟化。目前这些技术较为成熟，对这些内容感兴趣的读者可参阅《系统虚拟化——原理与实现》，本章对此不做详细分析。

本章重点关注的是 VMM 对外部设备的管理，而在外部设备中重点关注对 PCI 设备的管理。在一个处理器系统中，设置了许多专用硬件，如 IOMMU、PCIe 总线的 ATS 机制、SR-IOV（Single Root I/O Virtualization）和 MR-IOV（Multi-Root I/O Virtualization）机制，便于 VMM 对外部设备的管理。

13.1 IOMMU

在多进程环境下，处理器使用 MMU 机制，使得每一个进程都有独立的虚拟地址空间，从而各个进程运行在独立的地址空间中，互不干扰。MMU 具有两大功能，一是进行地址转换，将分属不同进程的虚拟地址转换为物理地址；二是对物理地址的访问进行权限检查，判断虚实地址转换的合理性。

在多数操作系统中，每一个进程都具有独立的页表存放虚拟地址到物理地址的映射关系和属性。但是如果进程每次访问物理内存时，都需要访问页表时，将严重影响进程的执行效率。为此处理器设置了 TLB（Translation Lookaside Buffer）作为页表的 Cache。如果进程的虚拟地址在 TLB 中命中时，则从 TLB 中直接获得物理地址，而不需要使用页表进行虚实地址转换，从而极大提高了访问存储器的效率。

从地址转换的角度来看，IOMMU 与 MMU 较为类似。只是 IOMMU 完成的是外部设备地

① 包括 SMP 系统和更为复杂的 NUMA 结构处理器系统。

② 本章出现的 IA 处理器是指 Intel 的 x86-64 处理器，而不是指 Itanium 处理器。

址到存储器地址的转换。我们可以将一个 PCI 设备模拟成为处理器系统的一个特殊进程，当这个进程访问存储器时使用特殊的 MMU，即 IOMMU，进行虚实地址转换，然后再访问存储器。在这个 IOMMU 中，同样存在 IO 页表存放虚实地址转换关系和访问权限，而且处理器为了加速这种虚实地址的转换，还设置了 IOTLB 作为 IO 页表的 Cache。单纯从这个角度来看，许多 HOST 主桥和 RC 也具备同样的功能，如 PowerPC 处理器的 Inbound 窗口和 Outbound 窗口，也可以完成这种特殊的地址转换。但是这些窗口仅能完成 PCI 总线域到一个存储器域的地址转换，无法实现 PCI 总线域到多个存储器域的转换。

目前设置 IOMMU 的主要作用是支持虚拟化技术，当然使用 IOMMU 也可以实现其他功能，如使“仅支持 32 位地址的 PCI 设备”访问 4 GB 以上的存储器空间。IA 处理器和 AMD 处理器分别使用 VT-d 和“IOMMU”，实现外部设备的地址转换。这两种技术都可以将 PCI 总线域地址空间转换为不同的存储器域地址空间，便于虚拟化技术的设计与实现。

13.1.1 IOMMU 的工作原理

根据虚拟化的理论，假设在一个处理器系统中存在两个 Domain，其中一个为 Domain 1，另一个为 Domain 2。这两个 Domain 分别对应不同的虚拟机，并使用独立的物理地址空间，分别为 GPA1（GPA 即 Guest Physical Address）和 GPA2 空间，其中在 Domain 1 上运行的所有进程使用 GPA1 空间，而在 Domain 2 上运行的所有进程使用 GPA2 空间。

GPA1 和 GPA2 采用独立的编码格式，其地址都可以从各自 GPA 空间的 0x0000-0000 地址开始，只是 GPA1 和 GPA2 空间在 System Memory 中占用的实际物理地址 HPA（Host Physical Address）并不相同，HPA 也被称为 MPA（Machine Physical Address），是处理器系统中真实的物理地址。而 PCI 设备依然使用 PCI 总线域地址空间，PCI 总线地址需要通过 DMA-Remapping 逻辑转换为 HPA 地址后，才能访问存储器。DMA-Remapping 逻辑的组成结构如图 13-1 所示。

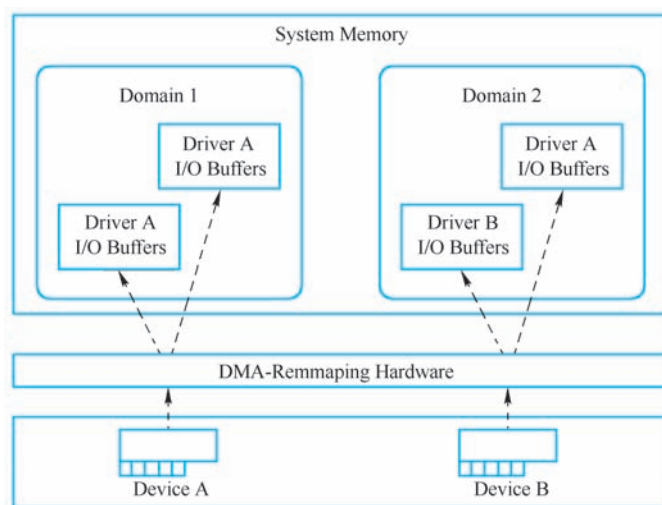


图 13-1 DMA-Remapping 的实现

在以上处理器模型中，假设存在两个外部设备 Device A 和 Device B。这两个外部设备分属于不同的 Domain，其中 Device A 属于 Domain 1，而 Device B 属于 Domain 2。在同一段时

间内，Device A 只能访问 Domain 1 的 GPA1 空间，也只能被 Domain 1 操作；而 Device B 只能访问 GPA2 空间，也只能被 Domain 2 操作。Device A 和 Device B 通过 DMA-Remapping 机制最终访问不同 Domain 的存储器。

使用这种方法可以保证 Device A/B 访问的空间彼此独立，而且只能被指定的 Domain 访问，从而满足了虚拟化技术要求的空间隔离。这一模型远非完美，如果每个 Domain 都可以自由访问所有外部设备当然更加合理，但是单纯使用 VT-d 机制还不能实现这种访问机制。

在这种模型之下，Device A/B 进行 DMA 操作时使用的物理地址仍然属于 PCI 总线域的物理地址，Device A/B 仍然使用地址路由或者 ID 路由进行存储器读写 TLP 的传递。值得注意的是虽然在 x86 处理器系统中，这个 PCI 总线地址与 GPA 地址一一对应且相等，但是这两个地址所代表的含义仍然完全不同。

GPA 地址为存储器域的地址，隶属于不同的 Domain，而 PCI 设备使用的地址依然是 PCI 总线域的物理地址，只是在虚拟化环境下，PCI 设备与 Domain 间有明确的对应关系。当这个 PCI 设备进行 DMA 读写时，TLP 首先到达地址转换部件 TA (Translation Agent)，并通过 ATPT[⊖] (Address Translation and Protection Table) 后将 PCI 总线域的物理地址转换为与 GPA 地址对应的 HPA 地址，然后对主存储器进行读写操作。其转换关系如图 13-2 所示。

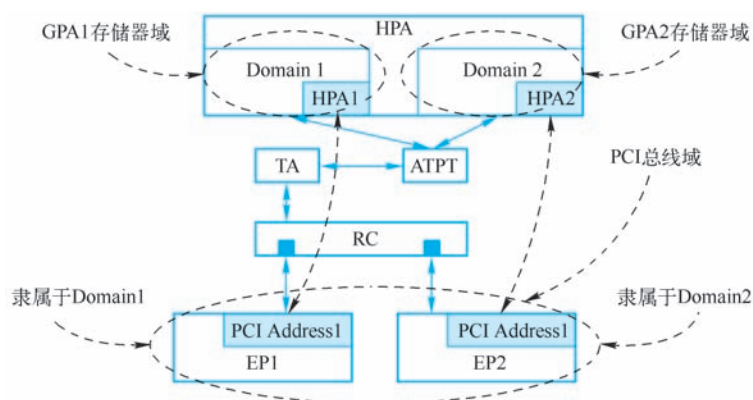


图 13-2 PCI 总线域物理地址与 HPA 的关系

在上图所示的处理器系统中，存在两个虚拟机，其使用的地址空间分别为 GPA Domain1 和 GPA Domain2。假设每个 GPA Domain 使用 1 GB 大小的物理地址空间，而且 Domain 间使用的地址空间独立，其地址范围都为 0x0000-0000~0x4000-0000。其中 Domain 1 使用的 GPA 地址空间对应的 HPA 地址范围为 0x0000-0000~0x3FFF-FFFF；Domain 2 使用的 GPA 地址空间对应的 HPA 地址范围为 0x4000-0000~0x7FFF-FFFF。在一个处理器系统中，不同的虚拟机使用的物理空间是隔离的。

在这个处理器系统中存在两个 PCIe 设备，分别为 EP1 和 EP2，其中 EP1 隶属于 Domain1，而 EP2 隶属于 Domain2，即 EP1 和 EP2 进行 DMA 操作时只能访问 Domain1 和 Do-

⊖ ATPT 相当于 I/O 页表，每一个 Domin 都具有独立的 I/O 页表。

main2 对应的 HPA 空间，但是 EP1 和 EP2 作为一个 PCIe 设备，并不知道处理器系统进行的这种绑定操作，EP1 和 EP2 依然使用 PCI 总线域的地址进行正常的数据传送。因为处理器系统的这种绑定操作由 TA 和 ATPT 决定，而对 PCIe 设备透明。在 EP1 和 EP2 进行 DMA 操作时，当 TLP 到达 TA 和 ATPT，经过地址转换后，才能访问实际的存储器空间。

下面以 EP1 和 EP2 进行 DMA 写操作为例，说明在这种虚拟化环境下，不同种类地址的转换关系，其步骤如下所示。

(1) Domain1 和 Domain2 填写 EP1 和 EP2 的 DMA 写地址和长度寄存器启动 DMA 操作。

其中 EP1 最终将数据写入到 GPA1 的 0x1000-0000~0x1000-007F 这段数据区域，而 EP2 最终将数据写入到 GPA2 的 0x1000-0000~0x1000-007F 这段数据区域。然而 EP1 和 EP2 仅能识别 PCI 总线域的地址。Domain1 和 Domain2 填入 EP1 和 EP2 的 DMA 写地址为 0x1000-0000，而长度为 0x80，这些地址都是 PCI 总线地址。

在 x86 处理器系统中，这个地址与 GPA1 和 GPA2 存储器域的地址恰好相等，但是这个地址仍然是 PCI 总线域的地址，只是由于 IOMMU 的存在，相同的 PCI 总线地址，可能被映射到相同的 GPA 地址空间，然而这些 GPA 地址空间对应的 HPA 地址空间不同。这个 PCI 总线地址仍然在 RC 中被转换为存储器域地址，并由 TA 转换为合适的 HPA 地址。

(2) EP1 和 EP2 的存储器写 TLP 到达 RC。

来自 EP1 和 EP2 存储器写 TLP 经过地址路由最终到达 RC，并由 RC 将 TLP 的地址字段转发到 TA 和 ATPT，进行地址翻译。

EP1 和 EP2 使用的 I/O 页表已经事先被 VMM 设置完毕，TA 将使用 Domain1 或者 Domain2 的 I/O 页表，进行地址翻译。EP1 隶属于 Domain1，其地址 0x1000-0000（PCI 总线地址）被翻译为 0x1000-0000（HPA）；而 EP2 隶属于 Domain2，其地址 0x1000-0000（PCI 总线地址）被翻译为 0x5000-0000（HPA）。值得注意的是在 TA 中设置了 IOTLB，以加速 I/O 页表的翻译效率，因此 TA 并不会每次都从存储器中查找 I/O 页表。

(3) 来自 EP1 和 EP2 存储器写 TLP 的数据将被分别写入到 0x1000-0000~0x1000-007F 和 0x5000-0000~0x5000-007F 这两段数据区域。

(4) Domain1 和 Domain2 都使用 0x1000-0000~0x1000-007F 这段 GPA 地址访问来自 EP1 和 EP2 的数据，这个 GPA 地址将转换为 HPA 地址，然后发向存储器控制器。在 IA 处理器系统中，使用 EPT 和 VPID 技术进行 GPA 地址到 HPA 地址的转换。

IA 处理器和 AMD 处理器使用不同的技术，实现 TA 和 ATPT。其中 IA 处理器使用 VT-d 技术，而 AMD 使用 IOMMU。从工作原理上看，这两种技术类似，但是在实现细节上，两者有较大区别。

13.1.2 IA 处理器的 VT-d

IA（Intel Architecture）处理器使用 VT-d 技术将 PCI 总线域的物理地址转换为 HPA 地址。这个映射过程也被称为 DMA Remapping。IA 处理器系统使用 DMA Remapping 机制可以辅助虚拟化技术对外部设备进行管理。

在 IA 处理器系统中，所有的外部设备都是 PCI 设备。每一个设备都唯一对应一个 Bus Number、Device Number 和 Function Number，为此 IA 处理器设置了一个专门的结构，即 Root

Entry Table^①，管理每一棵 PCI 总线树。在这种结构下，每一个 PCI 设备根据其 Bus、Device 和 Function 号唯一确定一个 Context Entry。VT-d 将这个结构称为“Device to Domain Mapping”结构，如图 13-3 所示。

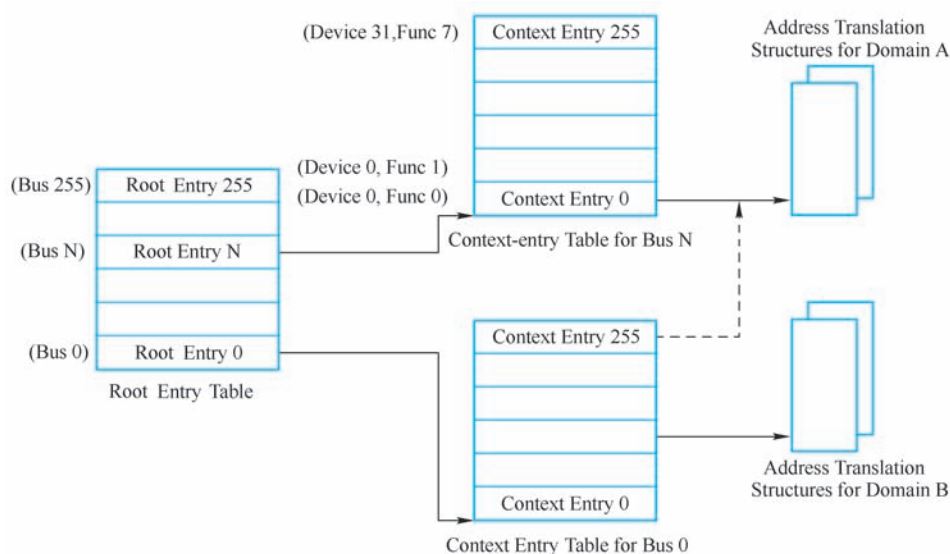


图 13-3 Device to Domain Mapping 结构

VT-d 一共设置了两种结构描述 PCI 总线树结构，分别为 Root Entry 和 Context Entry。其中 Root Entry 描述 PCI 总线，一棵 PCI 总线树最多有 256 条 PCI 总线，其中每一条 PCI 总线对应一个 Root Entry；每条 PCI 总线中最多有 32 个设备，而每个设备最多有 8 个 Function，其中每一个 Function 对应一个 Context Entry，因此每个 Context Entry 表中共有 256 表项。

在一个处理器系统中，一个指定的 PCI Function 唯一对应一个 Context Entry，这个 Context Entry 指向这个 PCI Function 使用的地址转换结构 (Address Translation Structures)。当一个 PCI Function 隶属于不同的 Domain 时，将使用不同的地址转换结构，但是在一个时间段里，PCI Function 只能使用一个地址转换结构，即 Context Entry 只能指向一个 Domain 的地址转换结构。这个地址转换结构的主要功能是完成 PCI 总线域到 HPA 存储器域的地址转换。

如图 13-1 所示，当一个设备进行 DMA 操作时，Domain 使用 PCI 总线域的地址填写这个设备和与 DMA 转送相关的寄存器。当这个设备启动 DMA 操作时，将使用 PCI 总线地址，之后通过 DMA-Remapping 机制将 PCI 总线地址转换为 HPA 存储器域地址，然后将数据传送到实际的物理地址空间中。而 Domain 通过处理器的 MMU 机制将 GPA 转换为 HPA，访问物理地址空间。

从图 13-3 中可以发现，每一个 Function 在每一个 Domain 中都可能有一个地址转换结构，以完成 GPA 到 HPA 的转换，因此在每一个 Domain 中最多有 256 个地址转换结构。这些结构无疑将占用部分内存，但是并不会产生较大的浪费。因为在实际设计中，同一个 Do-

① 如果处理器系统有多个 PCI 总线树 (Segment)，则需要设备多个 Root Entry Table。

main 下的所有 PCI 设备使用的总线地址到 HPA 地址的转换结构可以相同。因此在实现中，每个 Domain 仅使用一个地址转换结构即可。

IA 处理器使能 VT-d 机制后，PCI 设备进行 DMA 操作需要根据 Bus、Device 和 Function 号确定 Context Entry，之后使用图 13-4 所示的方法完成 PCI 总线地址到 HPA 地址的转换。

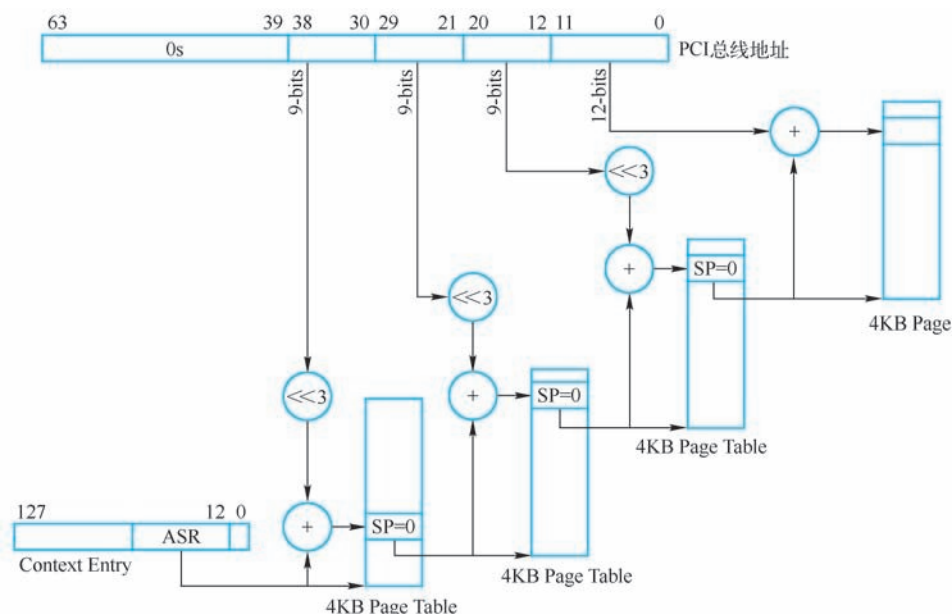


图 13-4 VT-d 使用的 PCI 总线地址到 HPA 的转换机制

在上图中，4 KB Page Table 中的每个 Entry 的大小为 8B，因此在计算偏移时，需要左移 3 位，PCI 总线地址通过 3 级目录，最终找到与 HPA 所对应的 4 KB 大小的页面，从而完成 PCI 总线地址到 HPA 的转换。值得注意的是，IA 处理器还支持 2 MB (SP=1)、1 GB (SP=2)、512 GB (SP=3) 和 1 TB (SP=4) 大小的 Super Page，而本节仅使用了 4 KB 大小的页面。

为了加快 PCI 总线地址到 HPA 地址的转换速度，IA 处理器分别为 Root Entry 和 Context Entry 设置了 Context Cache 以加快 Context Entry 的获取速度，同时还设置了 IOTLB 加速 PCI 总线地址到 HPA 地址的转换速度。

IOTLB 相当于 I/O 页表的 Cache，当一个 PCI 设备进行 DMA 操作时，首先在 IOTLB 中查找 PCI 总线地址与 HPA 地址的映射关系，如果在 IOTLB 命中时，PCI 设备直接获得 HPA 地址进行 DMA 操作；如果没有在 IOTLB 命中，则需要使用图 13-4 中所示的算法进行 PCI 总线地址到 HPA 地址的转换。

Intel 并没有公开“没有在 IOTLB 命中”的实现细节，当出现这种情况时，IA 处理器可能使用内部的 Microcode 完成图 13-4 所示的算法。使用 VT-d 除了可以有效地支持虚拟化技术之外，还可以支持一些只能访问 32 位地址空间的 PCI 设备访问 4 GB 之上的物理地址空间。

13.1.3 AMD 处理器的 IOMMU

AMD 处理器的 IOMMU 技术与 Intel 的 VT-d 技术类似，其完成的主要功能也类似。AMD 率先提出了 IOMMU 的概念，并发布了 IOMMU 的技术手册，但是 Intel 首先将这一技术在芯

片中实现。由于 AMD 和 Intel 使用的 x86 体系结构略有不同，因此 AMD 的 IOMMU 技术在细节上与 Intel 的 VT-d 并不完全一致。

AMD 处理器使用 HT (Hyper Transport) 总线连接 I/O Hub，其中每一个 I/O Hub 都含有一个 IOMMU，其结构如图 13-5 所示。

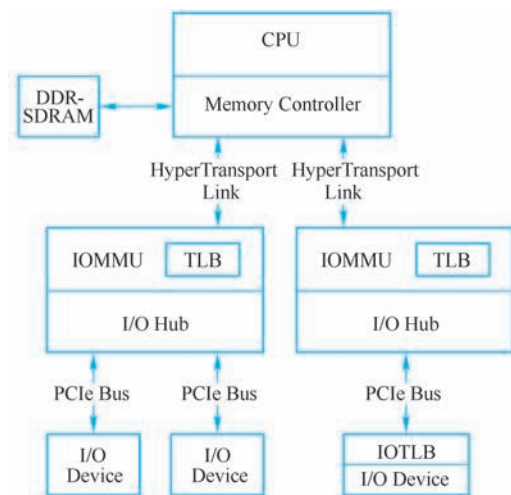


图 13-5 AMD 处理器的 IOMMU 结构

其中每一个 IOMMU 都使用一个 Device Table。AMD 处理器使用 Device Table 存放图 13-3 中的结构，Device Table 最多由 2^{16} 个 Entry 组成，其中每个 Entry 的大小为 256b，因此 Device Table 最大将占用 2 MB 内存空间，与 Intel 使用的 Root/Context Entry 结构相比，AMD 使用的这种方法容易造成内存的浪费。

在 I/O Hub 中的设备^①使用 16 位的 Device ID 在 Device Table 查找该设备所对应的 Entry，并使用这个 Entry，根据 I/O Page Table 结构最终找到 IO PTE 表，并完成 GPA 到 HPA 的转换。在 AMD 处理器中，GPA 到 HPA 的转换与图 13-4 中所示的方法有类似之处，但实现细节不同。IOMMU 使用一个新型的页表结构完成 GPA 到 HPA 的转换，这个页表结构基于 AMD64 使用的虚拟地址到物理地址的页表结构，但是做出了一些改动。AMD64 进行虚拟地址到物理地址的转换时使用 4 级页表结构，如图 13-6 所示。

与 Intel 处理器的结构类似，一个进程首先从 CR3 寄存器中获得页表的基地址指针寄存器“Page Map Level-4 Base Address”，之后通过 4 级索引最终获得 4 KB 大小的物理页面，完成虚拟地址到物理地址的转换。AMD 处理器也支持大页面方式，如果使用三级索引，可以获得 2 MB 大小的物理页面；使用二级索引，可以获得 1 GB 大小的页面。

IOMMU 使用的 I/O 页表结构基于以上结构，但是做出了一定的改动。在 IOMMU 中，4 级 I/O 页表指针可以直接指向 2 级 I/O 页表指针，从而越过第 3 级 I/O 页表，使用这种方法可以节省 Page Table 的空间。如图 13-7 所示。

① 其中 PCI 设备使用 Bus Number、Device Number 和 Function Number 组成 16 位的 Device ID，而 HT 设备使用 HT Bus Number 和 Unit ID 组成 16 位的 Device ID。

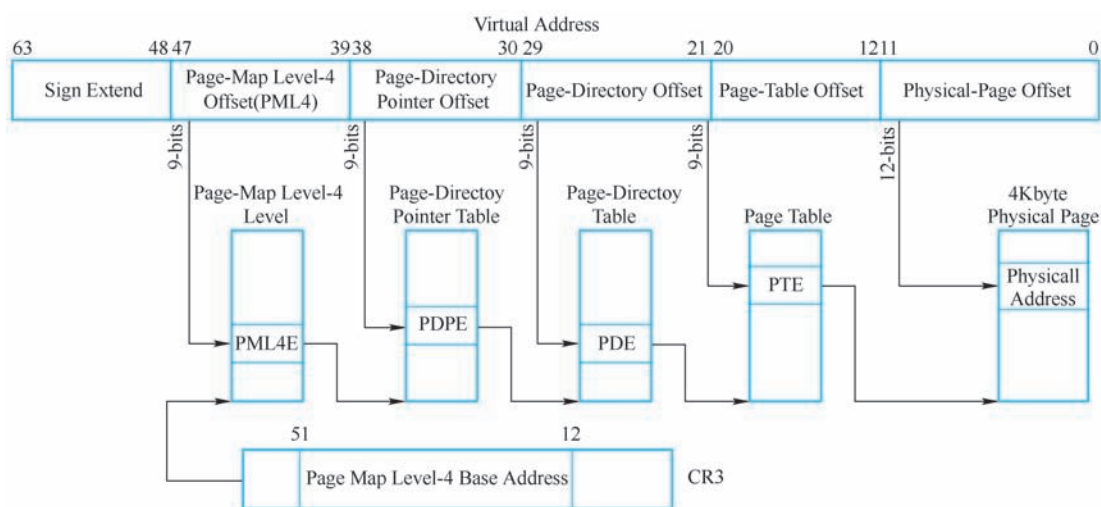


图 13-6 AMD64 虚拟地址到物理地址的页表结构

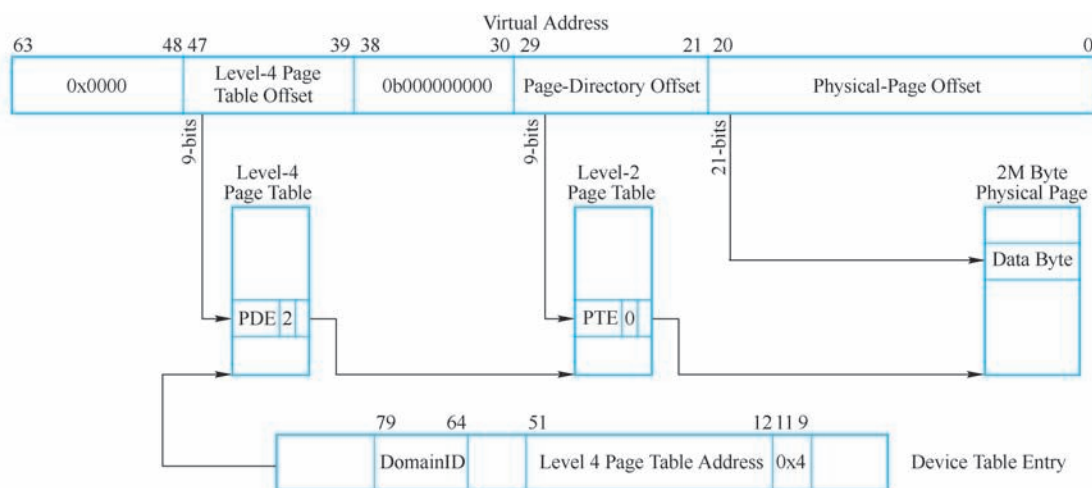


图 13-7 IOMMU 使用的 GPA 到 HPA 的转换机制

当设备进行 DMA 操作时，首先需要从相应的 Device Table 的 Entry 中获得“Level 4 Page Table Address”指针，并定位设备使用的 I/O 页表，最后使用多级页表结构，最终完成 PCI 总线地址到 HPA 地址的转换。Page Table 的 Entry 由 64 位组成，其主要字段如下所示。

- 第 51~12 位为 Next Table Address/Page Address 字段，该字段存放下一级页表或者物理页面的地址，该地址为系统物理地址，属于 HPA 空间。
- 第 11~9 位为 Next Level 字段，表示下一级页表的级数，其中在 Device Table 中存放的级数一般为 4，Level-N 级页表中存放的 Next Level 字段为 N-1~1。

如图 13-7 所示，在第 4 级页表的 Entry 中的 Next Level 字段为 2，表示第 4 级页表直接指向第 2 级页表，而忽略第 3 级页表。当该字段为 0b000 或者 0b111 时，表示下一级指针指向物理页面而不是页表。Next Level 字段为 0b000 时，表示所指向的物理页面的大小是固定的，AMD64 支持 4 KB、2 MB、1 GB、512 GB 和 1 TB (SP=4) 大小固定页面；如果 Next Level

字段为 0b111 时，表示所指向的物理页面大小是浮动的。如果 Level 2 Page Table 的 Entry 中的 Next Level 字段为 0b111，表示该 Entry 指向的物理页面大小浮动，其中物理页面大小和 GPA 的第 29~21 位相关，如表 13-1 所示。

表 13-1 Next Level 字段为 0b111 时的页面大小

29	28	27	26	25	24	23	22	21	Page Size	Default Page Size
Page Address									0	4 MB
Page Address								0	1	8 MB
Page Address							0	1	1	16 MB
Page Address						0	1	1	1	32 MB
Page Address					0	1	1	1	1	64 MB
Page Address				0	1	1	1	1	1	128 MB
...			0	1	1	1	1	1	1	256 MB
...	0	1	1	1	1	1	1	1	1	512 MB

AMD64 处理器使用这种 I/O 页表方式，可以方便地支持 4 KB、8 KB、…、4 GB 大小的浮动物理页面。除了 I/O 页表外，IOMMU 也设置了 IOTLB 以加快 GPA 到 HPA 地址的转换，这部分内容与 IA 处理器的实现方式类似，本章不对此继续进行描述。对 IOMMU 感兴趣的读者可以参考 AMD I/O Virtualization Technology Specification。

13.2 ATS (Address Translation Services)

单纯使用 IOMMU 并不能充分发挥处理器系统的效率，从图 13-2 中可以发现，所有 PCI 设备在进行 DMA 操作时，都需要经过 TA 和 ATPT 进行地址翻译，然后才能访问主存储器。因而 TA 和 ATPT 很容易成为瓶颈，从而影响虚拟化系统的整体效率。

除此之外，在图 13-2 中，EP1 和 EP2 分别隶属于 Domain1 和 Domain2。在正常情况下，一个 Domain 并不能访问其他 Domain 的 PCI 设备。但是如果处理器系统中存在一个恶意的虚拟机，而且 EP1 隶属于该虚拟机 (Domain1)。当 EP1 进行 DMA 写操作时，该虚拟机填写的 DMA 写地址可以与 EP2 的 BAR 地址空间重合，那么启动 DMA 写操作时，Domain1 可以将数据传递到 EP2，从而影响 Domain2 的正常运行。

解决这种异常的最合理的方法是，隶属于 Domain1 的 PCI 设备只能访问 GPA1 的空间，而仅使用 IOMMU 并不能解决该问题。解决该问题较为有效的方法是 PCI 设备进行数据传送的同时也进行地址转换，从而该 PCI 设备使用的地址是经过转换的 HPA 地址。此时再进行 DMA 写时，该数据将传递到与 Domain1 对应的 HPA 地址空间中，而不会将数据传送到 EP2。从而这个恶意的虚拟机并不会影响其他正常工作的虚拟机。

PCIe 总线使用 ATS 机制实现 PCIe 设备的地址转换。支持 ATS 机制的 PCIe 设备，内部含有 ATC (Address Translation Cache)，ATC 在 PCIe 设备中的位置如图 13-8 所示。

在 ATC 中存放 ATPT 的部分内容，当 PCIe 设备使用地址路由方式发送 TLP 时，其地址首先通过 ATC 转换为 HPA 地址。如果 PCIe 设备使用的地址没有在 ATC 中命中时，PCIe 设备将通过存储器读 TLP 从 ATPT 中获得相应的地址转换信息，更新 ATC 后，再发送 TLP。

与其他 Cache 类似，ATC 还可以被 Invalidate。当 ATPT 被更改时，处理器系统将发送 Invalidate 报文，同步在不同 PCIe 设备中的 ATC。

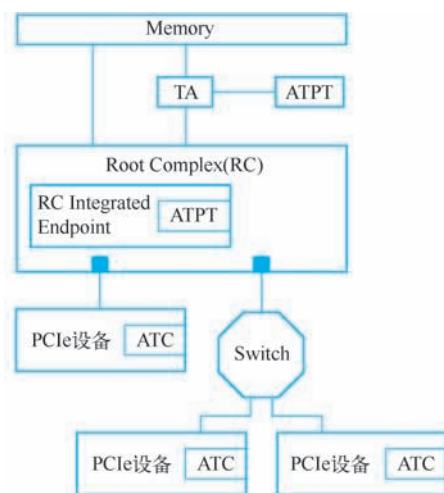


图 13-8 ATC 在 PCIe 设备中的位置

PCIe 总线在 TLP 中设置了 AT 字段以支持 ATS 机制。在 PCIe 总线中，只有与存储器相关的 TLP 支持 AT 字段。值得注意的是，只有处理器系统支持 IOMMU 时，PCIe 设备才可以使用 ATS 机制。

13.2.1 TLP 的 AT 字段

TLP 的 AT 字段与 ATS 机制直接相关。根据 AT 字段的不同，PCIe 设备可以发送三种类型的 TLP。

1. AT 字段为 0b00

当 AT 字段为 0b00 时，当前 TLP 的 Address 字段没有通过 ATC 进行转换，存放的是 PCI 总线域的物理地址。如果 PCIe 设备不支持 ATS 机制，而且处理器系统也没有使能 IOMMU 时，当前 TLP 的 Address 字段为 PCI 总线域的物理地址。PCIe 设备进行 DMA 操作时，该地址将被 RC 转换为存储器域的物理地址，然后对存储器进行读写操作。

如果 PCIe 设备不支持 ATS 机制，但是当前处理器支持 IOMMU 时，当前 TLP 的 Address 字段依然为 PCI 总线域的物理地址。PCIe 设备进行 DMA 操作时，该地址将被 TA 根据 I/O 页表的设置，转换为合适的存储器域物理地址。

如果当前处理器系统支持虚拟化技术，当前 PCIe 设备将隶属于某一个 Domain，此时该 PCIe 设备进行 DMA 操作时，数据将被传送到属于该 Domain 的存储器域中。

2. AT 字段为 0b01

当 AT 字段为 0b01 时，表示当前 TLP 报文为“Translation Request”报文。支持 ATS 机制的 PCIe 设备，必须支持这类报文。

该报文由 PCIe 设备通过存储器读请求 TLP 发出，其目的地为 TA。TA 收到该报文后，将根据 I/O 页表的设置，将合适的地址转换关系，通过存储器读完成 TLP，发送给 PCIe 设备。而 PCIe 设备收到这个地址转换关系后，将更新 ATC。

3. AT 字段为 0x10

当 AT 字段为 0x10 时，表示当前 TLP 的 Address 字段已经通过 ATC 进行地址转换。当 PCIe 设备使用存储器读写报文进行 DMA 操作，而 RC 收到这些报文时，将不再通过 TA 和 ATPT 进行地址转换，而直接将数据发送给存储器。从而减轻了 ATPT 进行地址转换的压力。

值得注意的是，经过 ATC 进行地址转换后，在 TLP 的 Address 字段中存放的依然是 PCI 总线域的物理地址，该物理地址为 HPA 地址在 PCI 总线域中的映像。

如果 TLP 中的 Address 字段没有经过 ATC 进行地址转换，而且处理器系统支持虚拟化技术，该地址为仍然对应 GPA 地址在 PCI 总线域中的映像，此时该 TLP 使用的 AT 字段为 0b00。这些地址在经过 RC 后，将被转换为存储器域的地址，然后进入 TA 和 ATPT 再次进行地址转换。由以上描述可以发现，PCIe 设备无论是否使用 ATC 机制，在 TLP 中存放的 Address 字段仍然保存的是 PCI 总线地址。

13.2.2 地址转换请求

PCIe 设备可以使用地址转换请求（Translation Requests）TLP 向 TA 提交地址转换请求。该 TLP 具有 64 位和 32 位两种地址格式。本节仅介绍 64 位地址格式，如图 13-9 所示。其中 AT 字段为 0b01 表示当前报文为地址转换请求 TLP。

该报文的格式与存储器读请求 TLP 的报文格式基本类似，但是在地址转换请求 TLP 中，一些字段的含义与存储器读请求 TLP 并不相同。该报文的作用是将 Untranslated Address 字段发送到 TA，而 TA 根据 ATPT 将 Untranslated Address 数据区域进行翻译，然后通过存储器读完成 TLP 将地址转换关系发送给 PCIe 设备。PCIe 设备收到这个存储器读完成 TLP 后将这个地址转换关系保存在 PCIe 设备的 ATC 中。

Untranslated Address 数据区域的长度由 Length 字段确定。在地址转换请求 TLP 中，Length 字段的最低位和高 5 位为 0，而且 Length 字段不能为 0b00-0000-0000，因此该地址转换请求 TLP 所访问的数据区域最小为 8B，而最大不能超过 RCB。而且该数据区域为 1DW 对界，First DW BE 与 Last DW BE 字段都为 0b1111。

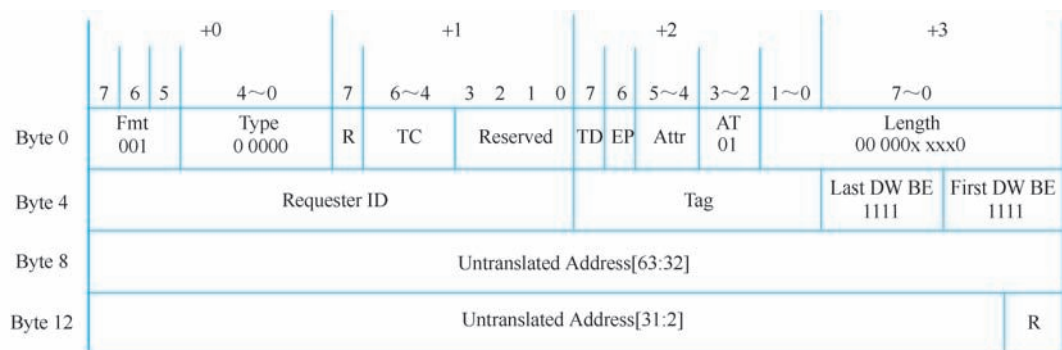


图 13-9 64 位地址转换请求 TLP 的格式

当 PCIe 设备与某个虚拟机绑定时，Untranslated Address 数据区域的 GPA 地址连续，但是其对应的 HPA 地址并不一定连续（在绝大多数虚拟机的实现中，为简化设计，GPA 所对应的 HPA 地址区域地址连续）。因此 PCIe 设备发送一个地址转换请求后，可能会从 TA 得

到多个地址转换关系，这些地址转换关系可以使用一个存储器读完成 TLP 发送给 PCIe 设备。在 PCIe 总线中，一个地址转换关系由 8B 组成，这也是地址转换请求 TLP 的 Length 字段至少为 0b10 的原因。

当 TA 收到地址转换请求 TLP 后，将查找 ATPT，然后通过存储器读完成 TLP，将转换关系发送给 PCIe 设备。如果地址转换成功时，TA 使用 CplD（带数据的存储器读完成报文）将转换关系发送给 PCIe 设备；否则使用 Cpl 将失败信息发送给 PCIe 设备。本节仅讨论 CplD 报文，其格式如图 13-10 所示。

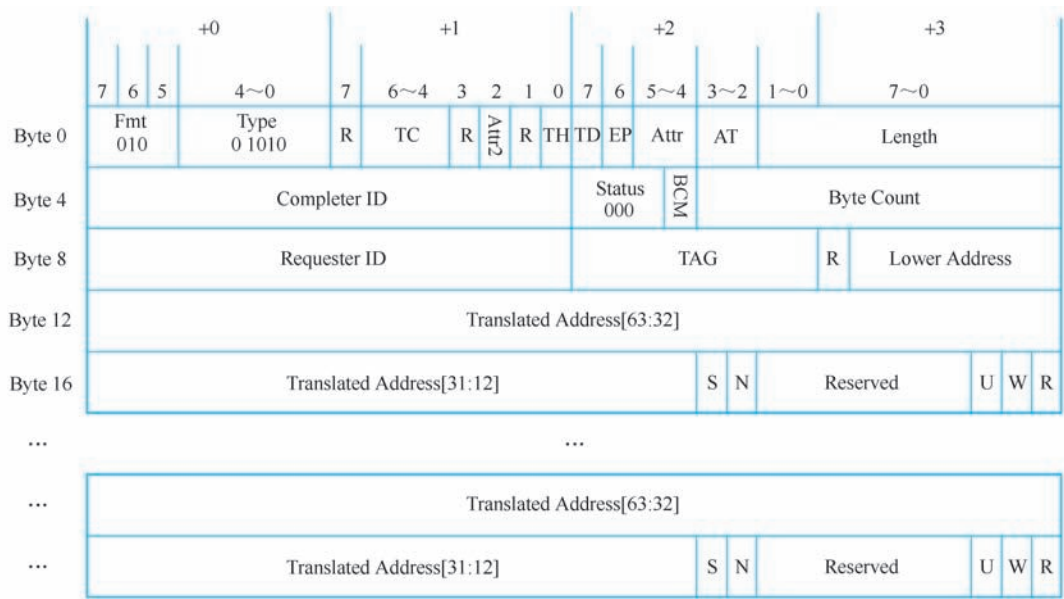


图 13-10 地址转换完成 TLP 的格式

地址转换完成 TLP 的报文头与存储器读完成报文头完全相同，而 Payload 字段由一个或者多个地址转换关系组成。一个地址转换关系由以下字段与位组成。

- Translated Address [63:12] 字段保存与 Untranslated Address 对应的 HPA 地址，即经过转换的地址。
- 而 U 位为 1 时，表示这段 HPA 空间只能使用 Untranslated 地址访问，即 PCIe 设备不能使用 AT 字段等于 0b10 的存储器读写 TLP；R 位为 1，表示 HPA 地址空间可读；W 位为 1 时，表示 HPA 地址空间可写。
- N 位为 1 时，PCIe 设备访问这段数据区域时，其存储器读写 TLP 的 No Snoop 位必须为 0，表示硬件需要进行 Cache 一致性操作；如果该位为 0，PCIe 设备将使用其他方法确定 No Snoop 位是否可以为 1。
- S 位需要与 Translated Address 字段联合使用，表示该段数据区域的大小，如表 13-2 所示。

由该表所示，Translated Address 数据区域的最小值为 4 KB，此时 S 位必须为 0。如果 S 不为 0，则表示这段数据区域大于 4 KB。当 Address31 为 0，而 Address [30:12] 和 S 位都为 1 时表示，这段区域为 4 GB，但是 4 GB 并不是 Translated Address 数据区域的最大值。

PCIe 设置还可以使用 Address [63:32] 字段，继续扩展数据区域的大小。

表 13-2 Translated Address 区域的大小

Translated Address																				S	页面大小/B		
63:32	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13			12	
X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	0	4 K	
X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	0	1	8 K
X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	0	1	1	16 K
...																							
X	X	X	X	X	X	X	X	X	X	X	X	X	0	1	1	1	1	1	1	1	1	1	2 M
...																							
X	X	X	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1 G
X	X	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	2 G
X	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	4 G

当 PCIe 设备支持 ATS 机制并进行 DMA 操作时，首先查找当前访问的地址是否在 ATC 中命中，如果命中则直接从 ATC 中获得 Translated Address，并使用 AT 等于 0b10 的存储器读写 TLP 与主存储器交换数据。TA 收到 AT 等于 0b10 的 TLP 后，将不使用 ATPT 进行地址转换，而将报文直接发送到存储器控制器，与主存储器进行数据交换。

如果 PCIe 设备访问的地址区域没有在 ATC 中命中时，PCIe 设备有两种处理方法，一是使用 AT 等于 0b00 的 TLP，即使用 Untranslated Address 直接访问存储器，这个 Untranslated Address 到达 TA 后，TA 根据 ATPT 的设置，将 Untranslated Address 进行地址转换，然后再将报文发送到存储器控制器中。

如果处理器系统中存在恶意的虚拟机时，PCIe 设备使用这种方法时将会带来安全隐患。因为恶意的虚拟机可以直接将这个 Untranslated Address 与其他 PCIe 设备使用的 BAR 空间重合，从而该虚拟机可以破坏隶属于其他虚拟机的 PCIe 设备，干扰其他虚拟机的正常运行，这是虚拟机系统禁止的行为。

因而当 PCIe 设备访问的地址区间没有在 ATC 中命中时，应该首先进行地址转换。采用这种方式时，PCIe 设备将首先向 TA 发送地址转换请求 TLP，并从 ATPT 中获得地址转换关系后，使用 TA 等于 0b10 的存储器读写 TLP，即使用 Translated Address 与主存储器进行数据交换，从而有效避免了上文所述的安全隐患。

目前尚无支持 ATS 机制的 PCIe 设备，但是通过本节的描述可以发现，使用 ATS 机制可以有效减轻 TA 进行地址转换的负担，同时避免虚拟机中存在的安全隐患。

13.2.3 Invalidate ATC

处理器系统更改 ATPT 时，需要使用 MsgD 报文通知相应 PCIe 设备，并 Invalidate ATC 中相应的 Entry，该 MsgD 报文也被称为“Invalidate Request Message”，其格式如图 13-11 所示。其中 PCIe 设备每收到一个 MsgD 只能 Invalidate ATC 中的一个 Entry。

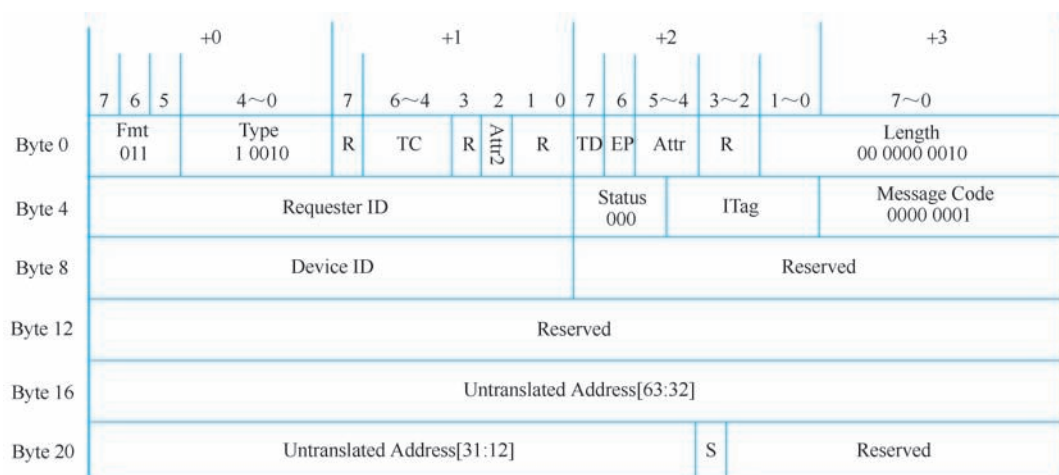


图 13-11 Invalidate Request Message 的格式

Invalidate Request Message 各个字段的描述如下所示。

- Fmt 字段为 0b011，表示报文头为 4DW，而且含有 Payload 字段。Length 字段为 0b10，表示该报文 Payload 的大小为 8B。而 TC、Attr、TD 和 EP 的含义与通用 TLP 头相同，详见第 6.1 节。
- Type 字段为 0b10010，表示该消息报文使用 ID 路由方式。其中 Requester ID 字段保存 TA 的 ID 号。Device ID 保存目标设备使用的 ID 号，即存放 ATC 的 PCIe 设备 ID，Message 报文使用该字段进行 ID 路由。
- ITag 字段与 Tag 字段的功能类似，取值范围为 0~31。当 TA 需要连续发送多个“Invalidate Request Message”报文时，使用该字段区别不同的 MsgD。

该 MsgD 报文的 Payload 字段中存放“Untranslated Address”，而 S 位用来表示数据区域的大小，如表 13-2 所示。当 PCIe 设备收到 Invalidate Request Message 报文后，根据 Untranslated Address 字段 Invalidate ATC 中对应的 Entry。PCIe 设备 Invalidate ATC 中对应的 Entry 之后，将向 TA 发送 Invalidate Completion Message 报文，表示已经 Invalidate ATC 中的对应 Entry，该报文的格式如图 13-12 所示。

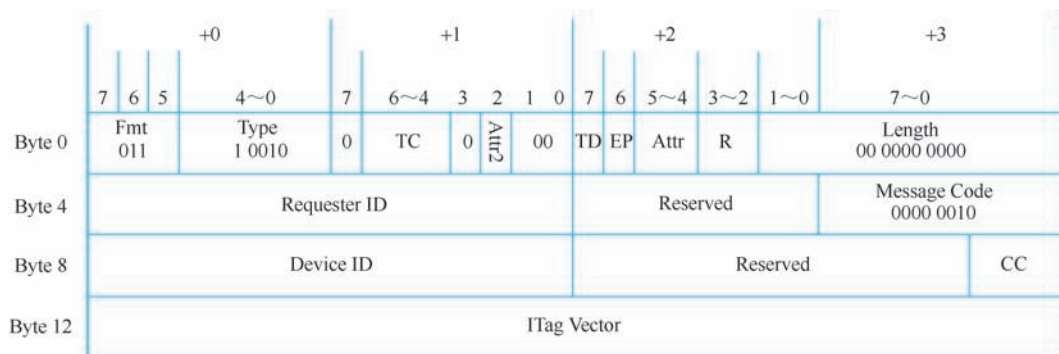


图 13-12 Invalidate Completion Message 的格式

Invalidate Completion Message 报文的各个字段的描述如下所示。

- Fmt、Type 等字段与 Invalidate Request Message 的对应字段类似。
- Requester ID 字段保存 TA 的 ID 号，而 Device ID 字段保存 PCIe 设备的 ID 号。
- CC 字段表示 PCIe 设备需要向 TA 发送 Invalidate Completion Message 报文的个数。当 CC 字段为 0 时表示需要 8 个这样的报文，为 n 时表示需要 n 个这样的报文。n 的最大值为 0x07。
- ITag Vector 字段由 32 位组成，其中每一位对应 Invalidate Request Message 报文的一个 ITag 字段，Invalidate Completion Message 通过 ITag Vector 字段可以向多个 Invalidate Request Message 报文发出回应，表示已经 Invalidate ATC 中的多个 Entry。

13.3 SR-IOV 与 MR-IOV

PCIe 总线除了提供了 ATS 机制外，还使用 SR-IOV 和 MR-IOV 机制，进一步优化虚拟化技术的实现。其中 SR-IOV 技术的主要作用是 将一个物理 PCIe 设备模拟成多个虚拟设备，其中每一个虚拟设备可以与一个虚拟机绑定，从而便于不同的虚拟机访问同一个物理 PCIe 设备。在 PCIe 体系结构中，即便使用了 ATS 和 SR-IOV 技术，在处理器系统中仍然只有一个 PCIe 总线域，所有的虚拟机共享这个 PCI 总线域，这为虚拟化技术的实现带来了不小的障碍。使用 SR-IOV 技术，可以解决单个 PCIe 设备被多个虚拟机共享的问题，但是并没有对管理 PCIe 设备的 Switch 进行约束。

提出 MR-IOV 技术的主要目的是解决多个处理器系统对一个 PCI 总线域共享的问题，其本质是 将一个物理 PCI 总线域，分解为多个虚拟的 PCI 总线域，多个处理器系统可以与多个 PCI 总线域对应，从而实现了不同 PCI 总线域间的隔离。MR-IOV 技术对 PCIe 总线进行了大规模的扩展，提出了 MRA（Multi-Root Aware）Switch、MRA Device 和 MRA RP（Root Port）的概念，同时对 PCIe 总线的数据链路层和流量控制进行了细微改动。

13.3.1 SR-IOV 技术

在 SR-IOV 技术没有引入之前，一个 PCIe 设备在一个指定的时间段内，只能与一个虚拟机（Domain 1）绑定，而其他虚拟机（Domain 2）访问与 Domain 1 绑定的 PCIe 设备时，需要首先向 Domain 1 发送请求，由 Domain 1 从 PCIe 设备获得数据后，再传送给 Domain 2。使用这种方法将极大增加在虚拟化环境下，虚拟机访问 PCIe 设备的延时，同时也干扰了其他虚拟机的正常运行。

而在处理器系统中并行设置多个同样的物理设备，不仅增加了系统成本，而且增加了处理器系统的规模，从而造成不必要的浪费。SR-IOV 技术在此背景下诞生。支持 SR-IOV 的 PCIe 设备，由多组虚拟子设备组成，其拓扑结构如图 13-13 所示。

基于 SR-IOV 的 PCIe 设备由多个物理子设备 PF（Physical Function）和多组虚拟子设备 VF（Virtual Function）组成。其中每一组 VF 与一个 PF 对应。在上图中存在 M 个 PF，分别为 PF0~M。其中“VF0, 1~N1”与 PF0 对应；而“VFM, 1~N2”与 PFM 对应。

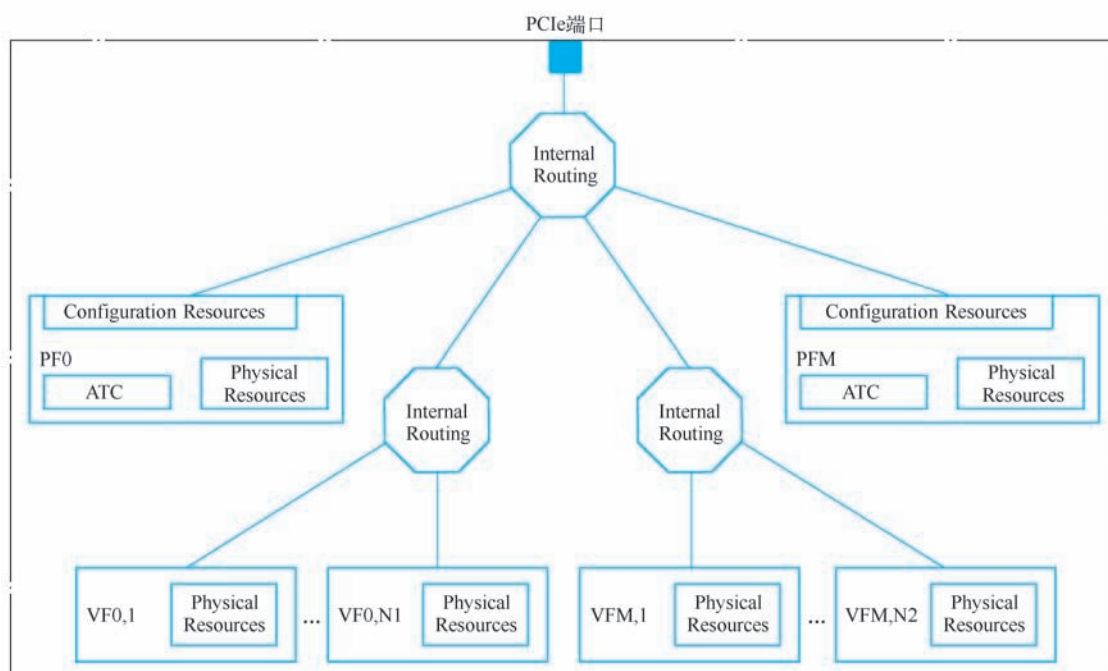


图 13-13 基于 SR-IOV 的 PCIe 设备

其中每个 PF 都有唯一的配置空间，而与 PF 对应的 VF 共享该配置空间，每一个 VF 都有独立的 BAR 空间，分别为 VF BAR0~5。从逻辑关系上看，这种做法相当于在一个 PF 中，存在多个虚拟设备。

在虚拟化环境中，每个虚拟机可以与一个 VF 绑定。假设在一个处理器系统中，网卡使用了 SR-IOV 技术，该网卡由一个 PF 和多个 VF 组成。其中每个虚拟机可以使用一个 VF，从而实现多个虚拟机使用一个物理网卡的目的。

PCIe 总线设置了 Single Root I/O Virtualization Extended Capabilities 结构以支持 SR-IOV 机制。对此感兴趣的读者可参阅 Single Root I/O Virtualization and Sharing Specification。这些细节对于非虚拟化技术的开发者并不重要。从本质上说，SR-IOV 技术与多线程处理器技术类似，只是多线程处理器技术应用于处理器领域，而 SR-IOV 将同样的概念应用于 PCIe 设备。

13.3.2 MR-IOV 技术

MR-IOV 技术的主要功能是将处理器系统的 PCI 总线域划分为多个虚拟 PCI 总线域，从而多个处理器系统可以共享同一个物理 PCI 总线域。MR-IOV 技术引入了几个新的概念，MRA RP、MRA Devices 和 MRA Switch。其中 MRA Switch 的结构如图 13-14 所示。

MRA Switch 与传统的 Switch 相比，其结构有较大不同。

- MRA Switch 由 0 个或者多个上游端口组成，如图 13-14 所示 MRA Switch 可以与多个 RP 连接，这个 RP 可以是 MRA RP 也可以是传统的 RP。在某些应用中，MRA Switch 可以作为中间节点与其他 MRA RP 相连，此时该 MRA Switch 不需要上游端口。
- MRA Switch 由 0 个或者多个下游端口组成，MRA Switch 可以与多个 MRA 设备连接，

也可以连接 SR-IOV 设备和传统的 PCIe 设备。MRA Switch 可以作为中间节点与其他 MRA Switch 相连，此时该 MRA Switch 可以不需要下游端口。

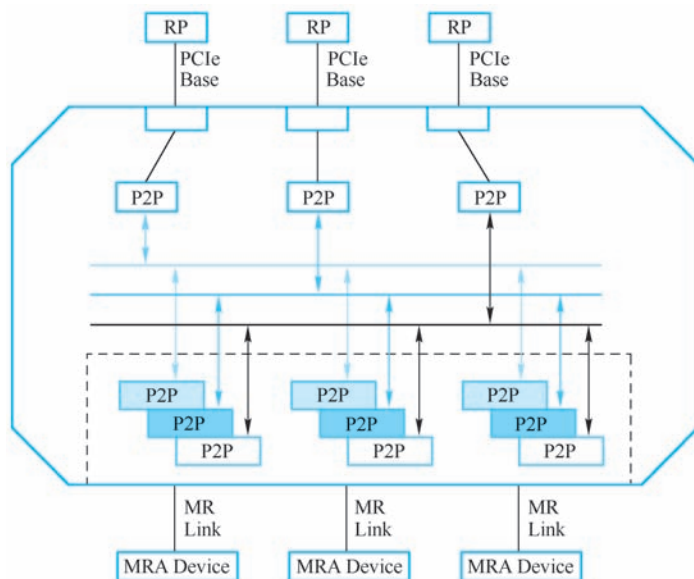


图 13-14 MRA Switch 的结构

- 使用 MRA Switch 可以组成多个虚拟 PCI 总线域 VHs (Virtual Hierarchies)。如图 13-14 所示，MRA Switch 由 3 套 P2P 桥组成，每一套 P2P 可以组成 1 个 PCI 总线域，这 3 个 PCI 总线域的地址空间独立。
- MRA Switch 可支持若干个双向端口，与其他 MRA Switch 和 MRA RP 相连。处理器系统之间可以使用这些双向端口组成复杂的服务器系统。

MRA Switch 的设计与实现较为复杂，而且该类芯片的应用范围较为有限，目前仅可能应用的支持虚拟化技术的高端服务器上。更为重要的是，Intel 并没有制作 Switch 芯片的传统，因此在短时间之内 MRA Switch 仅可能出现在 MR-IOV 规范中，而很难有实际的芯片。Intel 目前还没有支持 MRA RP 的 Chipset，但是可以预计 MRA RP 一定出现在 MRA Switch 之前，因而目前应该关注 MRA RP 与 MRA Devices 的连接拓扑结构。

MRA Devices 与 SR-IOV 设备相比略有不同，MRA Devices 略微更改了数据链路层。此外 MRA Devices 还重新定义了 SR-IOV 设备的 PF 和 VF，使得这些 PF 或者 VF 可以分属于不同的 PCI 总线域。

MRA Devices 可以与 MRA RP 或者 MRA Switch 联合使用，从而组成独立的 PCI 总线域。这些独立的 PCI 总线域可以与多个独立的虚拟机组成多个虚拟处理器系统。在这种结构下，不同的存储器域可以使用独立的 PCI 总线域，以最大限度地实现虚拟机对外部设备的隔离访问。MRA Device 的组成结构如图 13-15 所示。

在 MRA Device 中含有 1 个新的子设备 BF (Base Function)，该设备存放管理 MRA Devices 的 MR-IOV 的 Capability 结构，该结构用来管理在 MRA Device 的 PCI 总线域。除此之外，在该结构中还可以存放与设备相关的寄存器，如网卡使用的 MAC 地址。BF 使用“BF 0: f”进行描述，其中“0”表示 BF 使用的 VH 号，而“f”表示 BF 使用的 Function 号，其

值在 0~255 之间。值得注意的是 BF 使用的 VH 号只能为 0。

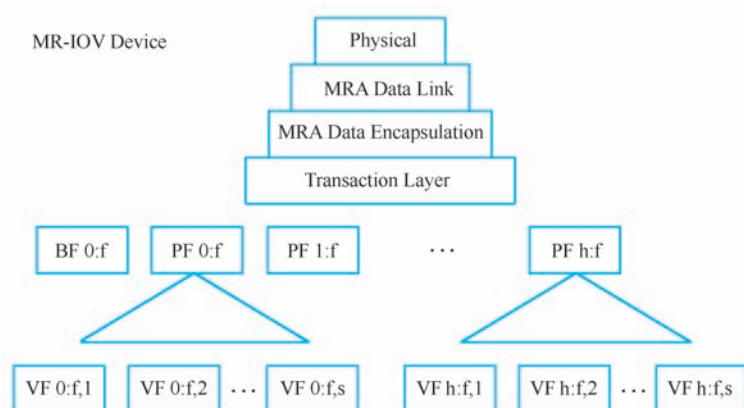


图 13-15 MRA Device 的结构

在 MRA Device 中，如果含有多个 PF 设备，这种 MRA Device 也被称为 SR-IOV/MRA Devices。这些 PF 设备使用“PF h: f”进行编码，其中“h”表示 PF 使用的 VH 号，而“f”表示 PF 使用的 Function 号，其值在 0~255 之间。

而与 SR-IOV 类似，在 MRA Device 中还存在多组 VF，其中每一组 VF 与一个 PF 对应，并使用“VF h: f, s”描述，其中“h”表示 VF 使用的 VH 号，而“f”表示 PF 使用的 Function 号，“s”表示 VF 号。

由以上描述可见，在 MRA Device 中，PF 和 VF 可以分别属于不同的虚拟 PCI 总线域 VH，并与 MRA RP 或者 MRA Switch 连接，形成一个完整的 PCI 总线域。为简便起见，本节仅介绍 MRA RP 与 SR-IOV 设备、SR-IOV/MRA Devices 和 MRA Devices 的连接关系，如图 13-16 所示。

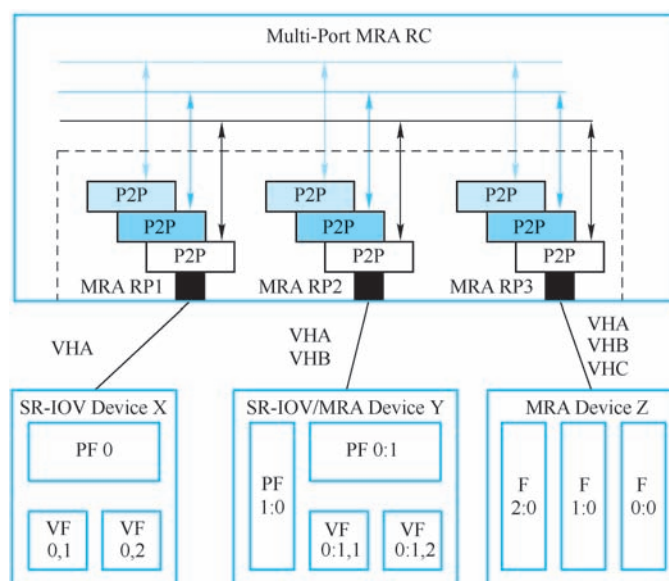


图 13-16 MRA RP 与 MRA Device 的连接

该图所示的 MRA RC 支持三个虚拟 PCI 总线域，分别为 VHA、VHB 和 VHC，并包含三个 MRA RP，其中 RP1 与 SR-IOV Device X、RP2 与 SR-IOV/MRA Device Y、RP3 与 MRA Device Z 连接。

Device X 仅含有 1 个 PCI 总线域，因此只能指派给一个虚拟 PCI 总线域，假设该设备使用 VHA；Device Y 中含有 2 个 PCI 总线域，假设该设备的 VH1 与 VHB 对应，而 VH0 与 VHA 对应；Device Z 中含有 3 个 PCI 总线域，假设该设备的 VH2 与 VHC 对应，VH1 与 VHB 对应，而 VH0 与 VHA 对应。

由此可知，在当前处理器系统中，虚拟 PCI 总线域 VHA 中包含 Device X 中的全部子设备、Device Y 中的“PF0:1”、“VF 0:1, 1”和“VF 0:1, 2”和 Device Z 中的“F0:0”；虚拟 PCI 总线域 VHB 中包含 Device Y 的“PF1:0”和 Device Z 中的“F1:0”；而虚拟 PCI 总线域 VHC 中包含 Device Z 中的“F2:0”。

假设在处理器系统中含有 3 个虚拟机，这 3 个虚拟机可以分别使用 VHA、B 和 C 这 3 个不同的虚拟 PCI 总线域，从而实现对 PCI 设备的隔离访问。MR-IOV 技术的实现细节较为复杂，本节对此不做深入介绍。

通过以上描述，可以发现使用 MR-IOV 技术，通过为虚拟机提供独立的 PCI 总线域，较好地解决了虚拟机对外部设备的隔离访问。而且处理器系统还可以使用 MRA Switch 组成更为复杂的网络拓扑结构，从而便于实现基于多个 SMP 系统的虚拟机。但是目前尚无支持 MR-IOV 技术的 RP 和 Switch。

13.4 小结

本章简单介绍了 PCIe 总线与虚拟化技术相关的内容。读者需要获得与处理器相关的虚拟化知识后，才能进一步理解这些内容。囿于篇幅，本章没有进一步介绍虚拟化技术的实现细节。

第 II 篇的内容到此告一段落，在本篇中较为详细地介绍了 PCIe 总线的层次结构，流量控制机制，电源管理、序和死锁以及虚拟化技术等一系列内容。本篇的内容并不局限于 PCIe 总线本身，希望读者可以从本篇中了解通用总线的设计与实现过程，以及值得注意的实现细节。